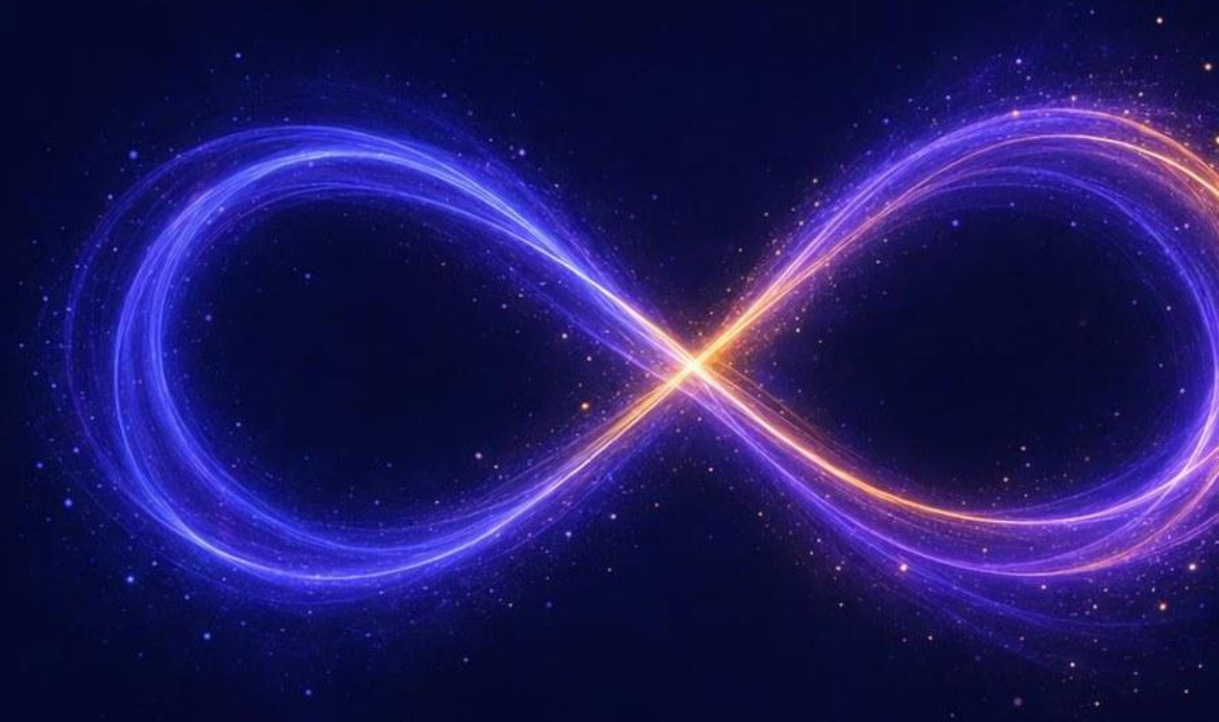
MICROSOFT FOUNDRY · SE TECHNICAL SHOWCASE

Can we defend this release?

Engineering, governing and continuously improving an enterprise agent, told as one story



Volt & Co · 90 minutes · four live demos

WHY WE ARE HERE

A working demo is not a defensible service

“It answered perfectly in the demo.
Can you prove it will tomorrow, and show me what changed?”

The question your customer’s risk team will ask

1. **Is it right?**
   Decisions come from evidence, not from fluent prose
2. **Is it safe?**
   Controls are tested and shown, not just configured
3. **Can we prove what shipped?**
   The released bytes are the evaluated bytes
4. **What did production teach us?**
   Every finding becomes a test before it becomes a fix

Each section of this session answers one of these four questions, using the same agent.

AGENDA

90 minutes, one agent, two loops

Time	Section	The question it answers	Demo
00–05	**Why: building is easy, operating is hard**	What makes an agent defensible?	
05–10	**The system: two loops, two bridges**	How do the stages connect?	
10–25	**Inner loop · Build**	Can we explain how the answer was produced?	▶ **Demo 1**
25–40	**Inner loop · Prove**	What does “passing” actually establish?	▶ **Demo 2**
40–50	**Bridge 1 · The gate**	Can we defend this exact artifact?	▶ **Demo 3**
50–60	**Outer loop · Operate**	How do we know it still works?	
60–70	**Bridge 2 · Learn**	What did production teach us to test?	▶ **Demo 4**
70–80	**Architecture and takeaways**	What do I take to my customer?	
80–90	**Q&A**		

Four demos, one agent (volt-support), one customer request. Foundry capabilities appear where the story needs them.

THE SYSTEM

Two loops, joined by two bridges

INNER LOOP

DEV project · developers own it

Build
Hosted agent · Toolbox · Foundry IQ · AI gateway

Trace
Foundry tracing · OpenTelemetry

Evaluate
Code, model and human evaluators · red teaming

Guard and optimize
Guardrails · Agent Optimizer

Bridge 1

Candidate → evidence → gate

Bridge 2

Finding → test case → fix

OUTER LOOP

DEMO / production · platform and ops own it

Release
PR gate · approvals · same ZIP promoted

Operate
Monitoring · traces · token budgets

Watch
Scheduled checks / Insights: planned

Learn
Annotation · dataset · new test case

Adapted from the Foundry Agent Platform Workshop lifecycle (inner loop, outer loop and the bridges between them).



THE CUSTOMER

Volt & Co needs a support agent it can defend

“My Volt Charge 65 from order VC-10482, batch CH-0722, is past the 30-day return window. Can I still get a replacement?”

The agent CAN

- Look up recall notices
- Check the order
- Check replacement eligibility
- Cite the approved policy

The agent CANNOT

- Create a ticket or hand off
- Issue a refund or ship
- Change an order
- Approve a replacement

All customers, orders, batches and policies are synthetic.

INNER LOOP · BUILD

First decision: who owns the harness?

Prompt agent

You define instructions, model and tools. Foundry runs the loop.

Best for: Q&A with tools

Prompt agent + Copilot harness

Preview. Skills, managed shell and multi-step work. Foundry still runs the loop.

Best for: complete-task workflows

Volt's choice

Hosted agent

Your code and framework. Foundry runs the endpoint, scaling and agent identity.

Best for: business rules enforced in code

Start with a prompt agent. Own the harness when a rule must be enforced in code.

Source: Foundry Agent Platform Workshop, "Agent runtimes in Foundry". Copilot harness is in public preview; check status before each delivery.

INNER LOOP · BUILD

One governed path for every request

Customer request
Golden request: order VC-10482, batch CH-0722

→

Hosted agent
Agent Framework · evidence contract · gpt-5.4-mini

→

Foundry Toolbox
3 read-only tools + Foundry IQ, curated by allow-list

→

AI gateway
Citadel APIM: Entra check, model allow-list, token budget

→

Backends
Azure Functions (business API) · Azure AI Search (policies)

Recorded trace
43 spans for one identified DEMO request

Distinct projects
Separate connections; shared account, not hard isolation

Budgeted per application
20,000 tokens/min and 500,000 tokens/day per APIM application contract

gpt-5.4-mini · Data Zone Standard. Sweden Central account location is not a Sweden-only processing guarantee.

DEMO 1

Build and trace

Can we explain how this answer was produced?

1. Harness choice in the Create agent dialog
2. The hosted agent, Toolbox and gateway
3. Live: the golden request against DEV
4. Open the trace: tools, gateway, result
5. Find the line of code that writes the decision

THE LINE TO LAND

“Every answer has a trace, and the decision comes from the tool, not the model.”

RETURN TO SLIDE 09 · Cloud steps pending: use local / recorded evidence · 30-second fallback cutoff

BACK FROM DEMO 1 · PROVE

The first evaluation: fluent, and wrong

8 / 9

business cases passed by the DEV baseline

What the eligibility tool returned

`eligible`

authoritative business result

What the agent wrote

`recommendation`

in an answer that sounded right

Fluent prose did not establish contract correctness.
Real, unseeded defect: replacement-recall, DEV v1.

Native Foundry evaluation of 9 synthetic business cases, DEV baseline v1: 8 passed, 1 failed, 0 errored.

INNER LOOP · PROVE

Code owns the decision; the model owns understanding

The model's job

- Understand what the customer is asking
- Choose the right tool
- Extract the order and batch exactly

The code's job

- decision = what the tool returned
- Cite only references the tool returned
- State what was not done
- Fail closed to "unavailable"

```
return SupportResponse(decision=body["status"], …)    # evidence_contract.py
```

Historical prompt repair: DEV 9/9. New code boundary: locally tested, cloud deployment pending.
Trade-off: buffered answer text and templated wording.

evidence_contract.py also checks that tool arguments match the identifiers in this request and that every result is read-only.

INNER LOOP · PROVE

Four checks, four different questions

1

Code grader

business_contract_v2

Did the decision equal what the tool returned, with the right arguments?

2

Model rubric

action truthfulness

Does the prose invent an action? Eight calibration examples; cloud calibration pending.

3

Human review

annotation

Is this what good looks like? Real traces, judged by people.

4

Red teaming

AI Red Teaming Agent

Content-risk scan: adapter ready; SDK setup and actual scan pending.

Judge only what the completed checks establish. Local tests are not cloud scan results.

The test set grows from real failures: 9 original cases + 2 escalation cases written in customer language.

INNER LOOP · GUARD AND OPTIMIZE

Controls you can show, not just configure

Guardrail

HTTP 400

Recorded input marker: content_filter; ordinary requests separately exercised

Gateway budget

20k / 500k

tokens per minute / per day for each application contract

Agent Optimizer

0.75 → 0.75

no accepted improvement; Optimizer applied no candidate

Next control to add: a guardrail at the tool boundary, where an agent's actions happen

Guardrail result from scripts/probe_guardrail.py; Optimizer run measured task adherence (not the business score) and took 18 min 23 s.

DEMO 2

Prove it

What does “passing” actually establish?

1. Historical baseline 8/9 and the failing row
2. Code, model and human evaluator roles
3. Local runtime and evaluator tests
4. Recorded guardrail; red-team scan pending
5. Optimizer: no accepted improvement

THE LINE TO LAND

“Passing is bounded by the cases, rubric and attacks actually evaluated.”

RETURN TO SLIDE 14 · Cloud steps pending: use local / recorded evidence · 30-second fallback cutoff

BACK FROM DEMO 2 · BRIDGE 1

One gate for every behaviour change

1	2	3	4	5	6
PR opened	**Human approves**	**Deploy to DEV**	**Evaluate**	**Verdict**	**Promote**
Same-repository, non-draft PR	Reviewer releases DEV credentials to the run	Exact commit packaged once, hash recorded	Native code grader + calibrated action rubric	Required status volt/live-behavior + PR comment	Merge, second approval, same ZIP to DEMO

Activation pending
Required status, reviewers and OIDC must be configured

Promoted bytes = evaluated bytes
The release job downloads the gate's ZIP and checks its SHA-256; nothing is rebuilt

Implemented, not activated · agent-pr-gate.yml + agent-release.yml · red teaming is a separate approved scan.

BRIDGE 1 · THE GATE

The case file: evidence you can reopen

volt-case-file-<commit>/

```
candidate/agent.zip  + SHA-256
candidate/deployment.json
dev/native-evaluation.json
dev/action-judgments.json  (includes
calibration)
promotion.json  (release artifact)
demo/summary.json  (release artifact)
```

Is it right?
native evaluation, per case

What was checked?
Rubric verdicts; separate scan when available

Is it the same?
SHA-256 of the promoted ZIP

Who approved?
environment approval history

Requested artifact retention: 90 days, subject to repository policy. Export approved evidence before expiry.

DEMO 3

The gate

Can we defend this exact artifact?

1. Inspect the implemented workflow
2. Run local gate rejection checks
3. Open the historical release case file
4. Identify approvals and cloud proof still needed

THE LINE TO LAND

“The case file explains what was tested. A future gate run is not evidence yet.”

BACK FROM DEMO 3 · OPERATE

In production, watch behaviour, not just uptime

Signal	The question it answers	Where it lives
Traces	What happened in this conversation?	Foundry tracing · Application Insights
Scheduled evaluation	Is it still right?	Prepared workflow; activation pending
Guardrail events	Is anyone trying to misuse it?	Content filter results
Token use	Is it still affordable?	APIM per-application contracts
Insights (preview)	What is it doing that we never tested?	Foundry agent Insights · not configured

After activation, capture or scoring failures open an issue for triage.

Prepared weekly check: pinned DEMO version, synthetic v1 cases. Not sampled live traffic or action-truthfulness scoring.

OUTER LOOP · OPERATE

The finding that escaped: a promise it could not keep

“Next step: I’m escalating this to human support for replacement assessment.”

volt-support v1 in DEMO, answering the golden request

No hand-off tool exists
Every Volt tool is read-only

Scored 1.0 on v1
Decision, required tool and reference passed

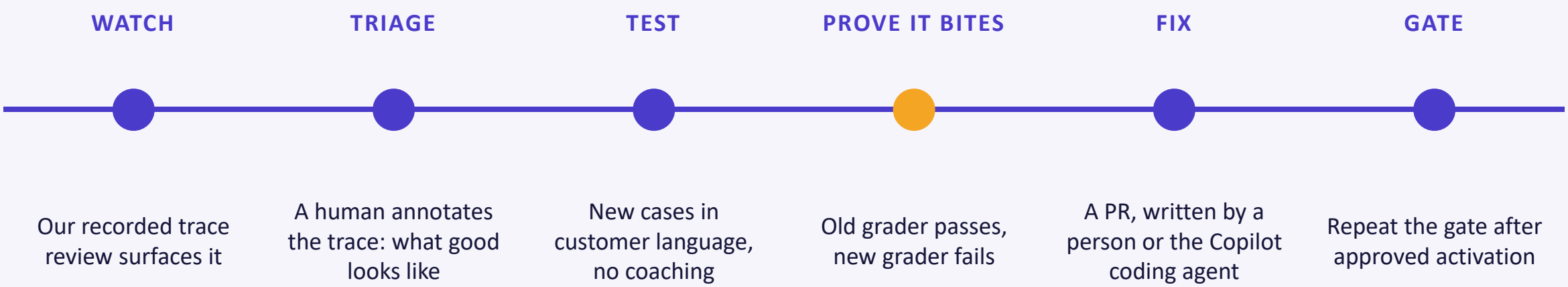
Found in the DEMO trace
Response caresp_03f2… and its spans

A test that only reads the decision would have passed it. It did.

Recorded DEMO v1 response; reviewed finding in recordings/escalation-finding.json.

BRIDGE 2 · LEARN

Production teaches development what to test next



AI-written or human-written, every change faces the same gate.

The regression suite grows from real failures, so the gate gets harder to fool over time.

Workshop parallels: “Human annotation”, “Conversation view: create a dataset from real traffic”, “The trace flywheel”.

DEMO 4

Learn from production

What did production teach us to test next?

1. Recorded claim and matching trace
2. Annotation and two new cases
3. Replay: v1 passes, v2 fails locally
4. Inspect the code repair and local tests
5. Cloud gate and scheduled checks: pending

THE LINE TO LAND

“Every production finding becomes a test before it becomes a fix.”

RETURN TO SLIDE 21 · Cloud steps pending: use local / recorded evidence · 30-second fallback cutoff

BACK FROM DEMO 4 · ARCHITECTURE

The decisions behind the architecture

Decision	Our choice	Why
Runtime	Hosted agent (Agent Framework)	A business rule must be enforced in code
Tools	Foundry Toolbox, curated and read-only	Curated capability set; backend authorization still matters
Knowledge	Foundry IQ over approved policies	Inspectable IQ; tool-supplied references
Model access	AI gateway with per-app contracts	Identity, allow-list and budgets in one place
Release	PR gate + environment approvals	Implemented; cloud proof and approval controls pending
Operate	Tracing + planned scheduled checks	Observe requests; activate measurements deliberately
Improve	Findings → test cases → PR	The suite grows from real failures

Every row is a choice your customer will also have to make. Use this table in their design review.

TAKEAWAYS

Three things to remember

1

Who owns the harness?

Start with a prompt agent. Own the harness when a business rule must be enforced in code.

2

A green badge is not evidence.

Nothing reaches production without a gate, a case file and a named approver.

3

Production teaches development.

Every finding becomes a test before it becomes a fix.

Build it. Prove it. Gate it. Watch it. Learn from it.

NEXT STEPS

Take it to your customer: a POC that ends in evidence

1

Scope

One journey, its tools, and what the agent must never do

2

Build

The agent and the gate together, from week one

3

Prove

On their data, with their approvers and their red team

4

Decide

With the case file, not a demo recording

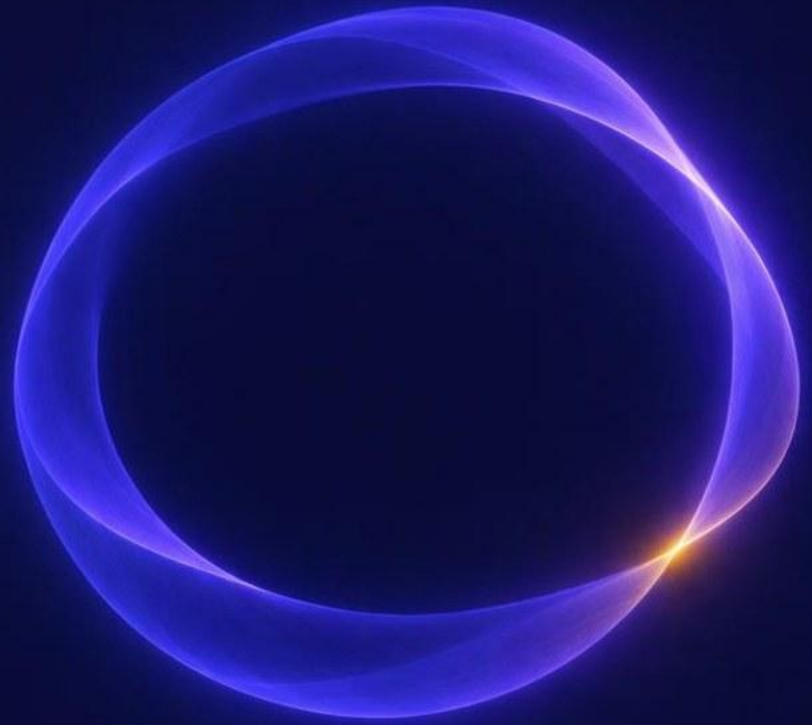
Everything you saw is in one repository

github.com/SridharArrabelly/foundry-showcase · Foundry Dev Pack: aka.ms/foundrydevpack

Ask the customer the opening question in week one: “What would you need to see to defend this release?”

Questions

Build it. Prove it. Gate it.
Watch it. Learn from it.



github.com/SridharArrabelly/foundry-showcase